COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

07

PACKAGE 8 · DESIGN REVIEW

Gate G capstone

A verified game-related digital subsystem

Undergraduate computer-engineering core

Modules 31-35 · Gate G

Edition: September 28, 2026

A guided self-study curriculum. Supplied examples are learning references, not completed learner projects. Simulation is the default; physical hardware is optional.

Contents

Page references and PDF bookmarks are linked for navigation.

The checkpoint contract 3
Recommended project: game command queue 3
Interface version GQ-1 3
Register map 4
Event and ownership policy 4
Development sequence and review gates 5
Minimum acceptance matrix 5
Evidence layers: what each result establishes 6
Assessment rubric: 100 points 7
Oral defense and live change 8
Optional board extension 8
Submission layout 8

The checkpoint contract

After Volume 7, implement one bounded game-related RTL subsystem. Valid choices include a synchronous command queue with a processor interface, a sprite compositor, a fixed-point vector unit or an equivalently scoped controller block. Deliver an interface specification, independent software reference, RTL, automated testbench, directed/random cases, assertions, reset review and synthesis reports.

The Module 34 processor model, RTL and architectural verification remain required even when the capstone uses a different subsystem. Gate G overlaps Module 35's project; it is not a second independent project added to the original 450-720-hour estimate. Physical deployment is optional. A simulator and synthesis tool are still required to complete the relevant evidence; Python tests alone do not pass this checkpoint.

Recommended project: game command queue

Build a 16-bit command FIFO behind a deliberately small memory-mapped interface. A simulated game thread submits command words, polls status, reads commands through a consumer transaction and handles a sticky event notification. This is a bounded device-contract exercise, not a complete console, GPU, DMA engine or coherent bus fabric.

Choose a queue depth D from 1 through 65535 for the declared contract. Verify D=1, D=3 and D=8 in simulation. Resource limits may make larger values impractical; supporting a parameter syntactically does not prove every instantiation meets a physical target. Reuse the conservative FIFO policy from Module 33: no empty bypass and no full same-edge replacement.

Keep your implementation in `work/gate_g/`, copied from the starter specification rather than overwriting the reference FIFO. Write the top-level software model independently of the RTL register decoding. The package supplies the lower-level FIFO example, not a finished capstone adapter or a passing student project.

Interface version GQ-1

Use one clock and synchronous active-high reset. Every request contains a 32-bit byte offset, a read/write flag and 32-bit write data. Request valid/ready determines request acceptance at the rising edge. Response valid/ready similarly determines response consumption. Response contains 32-bit read data and a one-bit error flag.

Allow only one outstanding transaction. Request-ready is asserted only when no response is outstanding. After an accepted request, register the result and assert response-valid in the following cycle. Hold response data/error stable until consumed. For simplicity, do not accept a replacement request on the same edge that the previous response is consumed. This extra bubble is an explicit contract choice, not an accidental performance bug.

Each accepted request performs its side effect **once**. A stalled response does not repeat a pop, push, flush or clear. The requester holds its complete request stable until acceptance and stops presenting that transaction afterward. Invalid offsets, misaligned addresses, unsupported writes and reserved nonzero bits return an error and cause no side effects.

Reset suppresses both acceptance channels, clears occupancy/pointers, outstanding response, pending event and IRQ enable. It cancels an unconsumed response and flushes commands. The host must discard the previous transaction epoch; no stale completion may return after reset. Reset is not a promise that an already consumed command can be undone in an external game system.

Register map

Offsets are relative to the device base. All accesses are 32-bit aligned and use the whole word; byte strobes are outside GQ-1.

Offset	Register	Read	Write
0x00	STATUS	bit0 empty; bit1 full; bits31:16 count; other bits zero	Error; no change
0x04	DATA	Pop and return old head in low 16 bits; empty returns error	Enqueue low 16 bits; full or nonzero high 16 bits returns error
0x08	CONTROL	Zero	bit0 flushes queue; any other set bit errors
0x0C	IRQ_PENDING	bit0 pending; other bits zero	bit0 is write-one-to-clear; reserved set bits error
0x10	IRQ_ENABLE	bit0 enable; other bits zero	Set enable from bit0; reserved set bits error

All error responses return read data zero. Successful writes return zero. A DATA read returns a zero-extended 16-bit word. STATUS reads snapshot pre-edge occupancy. CONTROL flush affects only queued commands, not an outstanding response from a different transaction, because transactions are serialized; it does not clear the pending interrupt.

Event and ownership policy

The event source is an accepted enqueue that changes occupancy from zero to one. Set pending even when IRQ output is disabled. Compute `pending_next = (pending AND NOT clear) OR event`; event wins over a simultaneous clear. In this serialized register interface, a clear and enqueue do not naturally occur in the same accepted request. Test the underlying event/clear primitive separately and document that distinction instead of inventing an impossible transaction.

IRQ is pending AND enable. It indicates an event since the last clear, not a count of commands or proof that the FIFO is currently nonempty. A flush may leave a pending notification. A single sticky bit coalesces repeated events; use an event counter only as an explicitly versioned extension.

Before request acceptance, the producer owns the word. After a successful enqueue, the device owns it. A full-error response means no enqueue occurred. After an accepted successful DATA read, the word is reserved by that transaction and no longer in the queue even if its response is backpressured. The response must retain the popped word until consumed or reset. This is why an output register is part of the adapter, not merely a convenient waveform feature.

Development sequence and review gates

G0 - freeze the contract. Write the register table, ready/valid timing, overflow policy, reset behavior, event semantics and error cases before RTL. Review a full-queue read, an empty read and a reset during response hold on paper.

G1 - executable reference. Implement a transaction-level queue/adapter model. Use abstract sequences and explicit ownership. Construct directed expected traces independently from the DUT. Add tests that prove the reference rejects invalid operations without changing state.

G2 - lower-level RTL. Verify the FIFO at D=1,3,8 with empty/full, simultaneous operations, stalls, wrap and reset. Read an actual waveform and explain pre-edge acceptance versus post-edge state. Record the simulator and seed.

G3 - interface integration. Add registered responses, address checks, exactly-once side effects and IRQ semantics. Run a software driver model against a simulated device. Preserve a command sequence end to end rather than checking only a final count.

G4 - structural implementation. Run synthesis, inspect inferred storage and unexpected latches, and save reports. Generic synthesis satisfies only the structural evidence tier. Any device-specific clock-frequency claim requires a selected part, complete constraints and the corresponding timing flow.

G5 - independent reproduction and defense. Have another person or a clean environment rerun the documented commands. Demonstrate one failed test and its fix. Complete the rubric and live review before marking the gate passed.

Minimum acceptance matrix

Each row needs a directed test ID, expected behavior and observed log/waveform location. Random tests supplement this matrix; they do not replace it.

ID	Scenario	Required result
G01	Reset from startup	Empty queue; IRQ/enable clear; no response
G02	One successful enqueue/read	Same word returned exactly once
G03	Empty DATA read	Error, zero response, state unchanged
G04	Write into full queue	Error; existing words preserved
G05	Depth-one fill/drain	Correct boundaries without bypass

ID	Scenario	Required result
G06	Non-power-of-two depth three	Pointers wrap at three, not four
G07	Multiple pointer wraps	Entire accepted-word order preserved
G08	Lower-level interior push and pop	Pop old head and append input
G09	Lower-level full push and pop	Only pop accepted; producer retains input
G10	Lower-level empty push and pop	Only push accepted; no phantom output
G11	Request backpressure	All request fields stable until accepted
G12	Response backpressure	Payload/error stable; side effect not repeated
G13	Back-to-back transactions	Declared one-outstanding bubble respected
G14	Invalid/misaligned offset	Error and no state change
G15	Write to STATUS	Error and no state change
G16	Reserved bits on DATA/control/IRQ	Error; no partial operation
G17	CONTROL flush with queued words	Queue empty; pending notification preserved
G18	Empty-to-nonempty event disabled	Pending sets; IRQ remains low
G19	Enable with pending already set	IRQ asserts without fabricating a new event
G20	Clear pending with no new event	Pending clears
G21	Event/clear primitive collision	New event survives; set-wins property
G22	Reset while request blocked	No unaccepted side effect
G23	Reset while response held	Response canceled; no stale completion
G24	Long randomized driver sequence	Every response and accepted-word order matches reference

Add invalid encoding/address/fault programs and pipeline/memory hazards to the separate processor evidence. Gate G's command-queue tests do not stand in for them.

Evidence layers: what each result establishes

Layer	Evidence	What it does not establish
Analytical/software model	Equations, source, tests, deterministic traces	RTL scheduling, metastability, physical timing

Layer	Evidence	What it does not establish
RTL simulation	DUT source, actual simulator log, waveforms, scoreboards	Exhaustive correctness or analog failure rates
Formal property check	Property, assumptions, engine, bounds/result, covers	Unstated properties, unconstrained real hardware
Generic synthesis	Netlist/cell report and warnings	Routed device resources, Fmax or power
Target implementation	Exact device, constraints, place/route and timing	Board signal integrity or product qualification
Physical experiment	Reviewed setup, instruments, measured conditions	Universal reliability or ASIC signoff

Record missing layers as not run or not claimed. Do not invent measurements to fill a form. In this package release, Python reference tests ran; HDL, SPICE, synthesis and formal execution did not.

Assessment rubric: 100 points

Area	Points	Full-credit evidence
Specification and ownership	15	Versioned, unambiguous interface and reset/error policy
Independent reference	10	Model independent of DUT structure; checked expected cases
RTL correctness and clarity	20	Synthesizable state/protocol implementation; documented parameters
Verification depth	25	Directed matrix, random seed, assertions, mutation, waveform explanation
Synthesis and timing interpretation	10	Actual synthesis log; no invented device-frequency claim
Integration and game relevance	10	Reproducible driver scenario and ordered command flow
Reproduction and oral defense	10	Clean rerun, live change, honest limitations

Suggested pass threshold is 80/100 with no critical blocker. A reviewer can demand revision above the numeric threshold when a central claim is unsupported. Core lesson and module prerequisites still apply.

Critical blockers: dropped/duplicated/reordered accepted work; repeated side effects under response stall; reset leaking stale commands/responses; nondeterministic uninitialized control; no actual RTL test execution; no actual synthesis report; a failed case hidden from the report; unsupported physical/ISA/ASIC claims; or omitted Module 34 processor work. A board purchase or demonstration is not a pass requirement.

Oral defense and live change

Explain a five-edge waveform with a full FIFO and a pending producer word. Name exactly which pre-edge signals define each transfer. Trace two nonblocking register assignments and contrast setup with hold timing. Explain why slowing a clock does not generally repair hold failure.

Then change D=3 to D=1 and predict which directed tests are most sensitive. Demonstrate reset while a response is stalled. Show one assertion and its assumptions, plus a trace that exercises its antecedent. Identify the strongest result that did not run and state what would be required to obtain it.

For the processor review, hand-encode one instruction, explain a branch offset relative to PC+1, locate the first retirement mismatch, and show why a skipped store must not commit. The reviewer should ask one small unprepared modification rather than accepting only a recorded demonstration.

Optional board extension

Before any physical work, document the exact board revision, power source, I/O voltages, clock source, constraints, pin mapping and reset circuit. Obtain a qualified low-voltage safety review. Begin with a non-destructive status output and known-good clock/reset; do not connect an arbitrary display or controller voltage directly to FPGA pins.

Physical bring-up follows verified simulation and target constraints. Save actual timing results and measurement conditions. No exposed mains, opened power supplies/displays, modified battery packs or unsupported power experiments are part of this course. A working FPGA display does not establish ASIC readiness. [R15, R33, V7-R01]

Submission layout

```
submission/
  README.md                # exact reproduction steps and limits
  spec/                    # GQ-1 or reviewed alternative contract
  model/                   # independent reference and driver
  rtl/                     # learner implementation
  tests/                   # directed, random and property checks
  evidence/                # raw logs, traces, versions and seeds
  synthesis/               # actual logs and reports
  processor/               # Module 34 completion evidence
  review/                  # rubric, defense and revision record
```

Include a provenance statement separating supplied code, your changes and outside sources. Passing means you can explain, change and reproduce the system, not merely redistribute the package.